ITS SURABAYA — INTELLIGENT TRANSPORTATION SYSTEMS

Task 2: DLH Waste Collection Simulation

Routing Optimization using the Directed Chinese Postman Problem (CPP)

1. Problem Definition & Graph Representation

The primary objective of this project is to simulate an optimal waste collection route for Department of Environment (DLH) dump trucks across the Surabaya road network. Operating within a user-specified radius x km from the central DLH depot, the truck must traverse every accessible street segment at least once while minimizing total travel distance and returning to the depot.

We model the urban road network as a directed graph (digraph) $G = (V, E)$. Vertices V represent street intersections, while directed edges E correspond to road segments. Employing a directed graph is mandatory to rigorously enforce one-way traffic regulations and turn restrictions; an undirected graph would produce invalid routes directing drivers against traffic. Edge weights reflect physical distances in meters computed using the Haversine formula on precise GPS coordinates.

Depot Projection: Since the DLH truck depot is defined via custom GPS coordinates rather than an existing map junction, the system automatically projects the depot location to the nearest node in the road graph using Euclidean/Haversine distance minimization.

2. Map Extraction, Data Storage, and Visualization

A. Map Data Extraction

To obtain an accurate road network, we query OpenStreetMap using the Python library `osmnx` (script: `Task2_download_data.py`). The network is extracted within a 16 km radius centered on Surabaya ($-7.2575, 112.7521$). To ensure routes remain practical for heavy waste collection trucks, we apply a custom OSM filter retaining only major urban road categories: `trunk` `primary` `secondary` `tertiary`. This filters out narrow alleys and pedestrian paths, reducing memory footprint while ensuring driver safety.

B. Data Storage Format

To guarantee zero latency and eliminate external API dependencies during simulation runs, pre-processed graph data is stored locally in a structured JSON file (`surabaya_map.json`). It comprises three primary dictionaries:

- `nodes`: Maps each intersection ID to its GPS coordinates `[latitude, longitude]`.
- `edges`: An adjacency list representing directed road segments and physical lengths in meters.
- `edge_names`: Stores official street names for every edge, enabling human-readable navigation directions.

C. Map Display & Interactive Dashboard

The user interface is constructed using **Streamlit** and **Folium** (script: `Task2_Chinese_Postman.py`). It renders high-contrast *Esri World Street Map* tiles centered on the DLH depot. The depot is highlighted with a green truck icon, surrounded by a semi-transparent red circle representing the selected collection radius (x km). Operators can enter coordinates manually or click directly on the interactive map, which dynamically updates the depot location via `st.rerun()`. The resulting Eulerian route is plotted as a continuous red overlay (`folium.PolyLine`).

3. Algorithm Explanation

A. Subgraph Extraction & Connectivity Isolation

Nodes within radius x from the depot are selected using Haversine filtering. We then run Dijkstra's algorithm from the depot node to isolate the strongly/weakly connected component accessible from the depot, discarding isolated or unreachable road segments.

B. Graph Eulerianization (Degree Balancing)

A directed graph contains an Eulerian circuit if and only if it is connected and balanced: every vertex must have equal in-degree and out-degree ($in(v) = out(v)$).

1. **Imbalance Identification:** For all nodes, we evaluate degree imbalance $in(v)$ - $out(v)$. Vertices with excess outgoing edges form the source set, while those with a deficit form the target set.
2. **Greedy Path Duplication:** Dijkstra's algorithm finds the shortest path between imbalanced nodes. The edges along these paths are duplicated in the graph until all nodes satisfy $in(v) = out(v)$, converting the graph into a Eulerian digraph.

C. Eulerian Circuit Generation (Hierholzer's Algorithm)

Once balanced, **Hierholzer's algorithm** constructs the circuit. Starting from the depot, an explicit stack-based depth-first search traverses every unused edge. Sub-cycles are merged into the main circuit until all edges are visited exactly once. The sequence of nodes is cross-referenced with `edge_names` to print sequential street-by-street instructions.

4. Algorithmic Complexity Analysis

Let V be the number of intersections and E the number of road segments in the extracted subgraph.

Algorithmic Component	Time Complexity	Space Complexity	Primary Role
Dijkstra's Algorithm	$O((V + E) \log V)$	$O(V + E)$	Connectivity isolation & shortest path duplication.
Hierholzer's Algorithm	$O(E)$	$O(V + E)$	Eulerian circuit traversal & path generation.
Total System Pipeline	$O(K \cdot (V + E) \log V)$	$O(V + E)$	Overall graph balancing & Eulerian pathing (K iterations).